

Pipeline Workflow Framework for Demo and Migration Comparison

This document defines the workflow framework to explain, build, and compare the legacy pipeline pattern and the future dbt-based pipeline pattern for the demo environment. It is designed to act as a reusable reference for implementation, explanation, rehearsal, and later comparison across both pipeline styles.[cite:15] [cite:42]

Purpose

The framework exists to make the end-to-end process easy to understand for internal teams and stakeholders by showing how data enters the platform, how it moves through each layer, where business logic lives, how validation is performed, and how the legacy and future approaches differ.[cite:42] [cite:54]

The target operating model uses Matillion primarily for extraction and loading, while dbt structures transformation logic into staging, intermediate/curated, and marts/DW layers. This separation aligns with dbt project-structure guidance and supports modularity, testing, and clear lineage.[cite:7] [cite:15] [cite:56]

Framework scope

The workflow framework covers four views of the same solution so the team can explain it from multiple angles:

- **Dataflow view** — where data physically moves from source to final consumption.[cite:42] [cite:54]
- **Workflow view** — what sequence of tasks and jobs run, and in what order.[cite:45] [cite:46]
- **Control view** — where validation, testing, freshness checks, and approvals happen. [cite:48] [cite:53]
- **Comparison view** — how old and new pipelines are mapped and reconciled for migration proof.[cite:26] [cite:15]

Architecture views

Legacy pipeline view

The legacy demonstration pipeline represents the current-state pattern where raw data is loaded and then transformed in a tool-driven flow, with transformation steps grouped into legacy jobs that populate a Data Lake or legacy transformed layer before delivery to reporting outputs.[cite:15] [cite:56]

```
Source System(s)
-> Matillion Extract + Load
-> RAW / Landing
-> Matillion Transformation Jobs
-> DL_LEGACY / Legacy Transformed Layer
-> Legacy Output / Reporting Table
```

Future pipeline view

The future-state pipeline uses the layered dbt pattern in which source tables are declared in dbt, then transformed through staging, intermediate, and marts layers before being published to the DW layer for reporting and consumption.[cite:7] [cite:42] [cite:54]

```
Source System(s)
-> Matillion Extract + Load
-> RAW / Landing
-> dbt source definitions
-> STAGING models
-> CURATED / INTERMEDIATE models
-> DW / MART models
-> Consumer-facing output
```

Standard pipeline components

Every pipeline in the demo should be described using the same component model so that comparison is easy later.[cite:42] [cite:54]

Component	Legacy pipeline role	Future pipeline role
Source	Origin system such as DBMS, file, API, or planning tool.[cite:15]	Same source system, unchanged.[cite:15]
Ingestion	Matillion extracts and loads data into RAW. [cite:46][cite:56]	Matillion extracts and loads data into RAW. [cite:46][cite:56]
Raw storage	Landing/RAW holds source-faithful data with minimal change.[cite:42]	Landing/RAW holds source-faithful data with minimal change.[cite:42]
Transform layer	Matillion jobs perform cleansing, joins, business rules, and output shaping.[cite:15] [cite:56]	dbt models perform staged cleansing, business logic, and final marts.[cite:7] [cite:54]
Quality controls	Usually embedded in job logic or manual validation.[cite:26]	dbt tests, source freshness, docs, and lineage are explicit and repeatable.[cite:48][cite:53]
Output	Legacy transformed tables or reporting-ready tables.[cite:15]	DW/mart tables for BI and downstream users. [cite:7][cite:54]

Common workflow blueprint

Each pipeline, whether simple or moderate, should follow the same blueprint so it can be explained consistently to others.[cite:42][cite:54]

Step 1: Source onboarding

- Identify source system, source owner, refresh frequency, and business purpose.
- Define source objects: tables, files, APIs, extracts, and expected keys.
- Record expected grain, example records, and SLA for update frequency.[cite:48][cite:53]

Step 2: Raw ingestion

- Matillion loads source data into the RAW schema using source-aligned table names.
- Minimal changes are allowed here: audit columns, load timestamp, file metadata, and source-run identifiers.[cite:15][cite:46]
- RAW is the baseline shared input for both legacy and future pipeline variants.[cite:42]

Step 3: Legacy transformation path

- Matillion legacy transformation jobs read from RAW.
- Business rules, joins, filters, mappings, and aggregations are implemented in job steps.
- Outputs are written to DL_LEGACY or a legacy-style output schema.[cite:15][cite:26]

Step 4: Future dbt transformation path

- dbt `source()` definitions point to RAW tables so the transformation path starts from the same source baseline.[cite:42][cite:52]
- Staging models standardize names, types, null handling, and data quality conventions.[cite:7][cite:54]
- Intermediate/curated models apply joins, business rules, calculations, enrichment, and reusable logic.[cite:2][cite:44]
- Marts/DW models publish consumer-facing fact and dimension outputs.[cite:7][cite:37]

Step 5: Validation and quality control

- Source freshness should be defined for key RAW tables using a loaded timestamp or supported freshness strategy, and run at a frequency aligned to SLA expectations.[cite:48][cite:55][cite:57]
- dbt generic tests such as `not_null`, `unique`, `relationships`, and `accepted_values` should be applied to important staging and downstream models.[cite:3]
- Legacy and future outputs should be reconciled through counts, totals, sample record checks, and exception analysis.[cite:26]

Step 6: Publishing and consumption

- Final outputs should be published to a clearly named DW/mart schema for reporting and demo consumption.[cite:7][cite:54]
- Demo queries, screenshots, and stakeholder walkthroughs should use the final output layer rather than intermediate processing layers.

Layer-by-layer framework for future dbt pipelines

RAW / Landing

Purpose: preserve source-loaded data with minimal change so it acts as the auditable foundation for all downstream processing.[cite:42][cite:54]

Rules:

- One raw table per source object where practical.
- Add audit columns such as `load_ts`, batch ID, or file name when available.
- No business logic here.[cite:42]

STAGING

Purpose: convert raw source data into clean, typed, consistently named source-aligned models.[cite:7][cite:54]

Rules:

- One staging model per major source table.
- Perform renaming, casting, trimming, standard flags, and light deduplication only.
- No multi-source business joins unless there is a compelling exception; use intermediate models instead.[cite:44]

Naming pattern:

```
- `stg_<source>__<entity>`. [cite:7]
```

CURATED / INTERMEDIATE

Purpose: hold reusable business logic and integration steps between source-aligned staging and final marts.[cite:2][cite:44]

Rules:

- Use for joins, conformance, derivations, mappings, filtered subsets, and reusable calculations.
- Break complex logic into more than one intermediate model if that makes lineage and testing clearer.[cite:44]

Naming pattern:

```
- `int_<domain>_<purpose>`.
```

DW / MARTS

Purpose: publish final business-facing models for analytics, BI, and demo reporting.[cite:7]
[cite:37]

Rules:

- Final facts and dimensions live here.
- These models should be documented, tested, and easy to explain to business users.
- Demo dashboards and comparisons should read from this layer.[cite:7]

Naming pattern:

```
- `dim_<entity>`, `fct_<process>`, `mart_<use_case>`.
```

Pipeline framework templates

Template A: Simple pipeline

Use this for a DBMS or operational provisioning example with low transformation complexity.

```
Source DB Table
-> Matillion extract job
-> RAW.provisioning_raw
-> Legacy path: Matillion job -> DL_LEGACY.provisioning_legacy
-> Future path: stg_dbms__provisioning
    -> int_provisioning_enriched
    -> fct_provisioning
```

Recommended explanation points:

- Show the same raw input feeding both old and new paths.
- Emphasize how dbt splits logic into readable layers.
- Use this pipeline to teach the framework basics.[cite:54]

Template B: Moderate pipeline

Use this for Anaplan or planning-style data with more mappings, joins, and business rules.

```
Source Planning Extracts + Reference Tables
-> Matillion extract/load jobs
-> RAW.anaplan_plan_raw + RAW.reference tables
-> Legacy path: Matillion transformation workflow -> DL_LEGACY.plan_legacy
```

```

-> Future path: stg_anaplan__plan
                -> stg_ref__mapping
                -> int_plan_mapped
                -> int_plan_enriched
                -> fct_plan
                -> dim_scenario / dim_region

```

Recommended explanation points:

- Show why the intermediate layer matters for more complex logic.[cite:2][cite:44]
- Highlight that complex logic becomes easier to trace when split into models with lineage.[cite:7][cite:50]

Workflow run sequence

Use the same job sequence for every demo run so the audience can follow a stable pattern.[cite:45][cite:46]

Sequence	Job / Activity	Tool	Output
1	Load source data to RAW	Matillion	Refreshed RAW tables.[cite:46]
2	Optional source freshness check	dbt	Freshness pass/warn/fail view for source timeliness.[cite:48][cite:53]
3	Run legacy transformation job	Matillion	Updated legacy output tables.[cite:15]
4	Run dbt staging and downstream models	dbt Cloud	Updated STAGING, CURATED, and DW models.[cite:42][cite:45]
5	Run dbt tests	dbt Cloud	Validation results for key models.[cite:3]
6	Generate docs / lineage if needed for demo	dbt Cloud	Visual lineage and model documentation.[cite:45][cite:50]
7	Run comparison queries	Snowflake	Side-by-side reconciliation outputs.
8	Present results	Demo walkthrough	Stakeholder-ready narrative.

Control and validation framework

Data controls

- Freshness checks on critical source tables should run frequently enough to reflect SLA expectations; dbt guidance recommends running freshness jobs at least twice as often as the shortest SLA.[cite:48]
- Audit columns in RAW help trace what batch or file created each load.[cite:42]

Model controls

- Apply generic tests in staging and downstream models for keys, relationships, nulls, and allowed values.[cite:3]
- Keep failed test outputs or screenshots available for demo explanation if you want to show governance behavior.

Reconciliation controls

For every legacy-vs-future pipeline pair, create a repeatable reconciliation pack with:

- Row count comparison.
- Aggregated metric comparison.
- Distinct key comparison.
- Sample record comparison.
- Exception table for mismatches.[cite:26]

Comparison framework

The comparison framework is what allows the team to prove migration readiness and explain the difference between approaches clearly.[cite:26] [cite:15]

Comparison dimensions

Dimension	Legacy pipeline	Future dbt pipeline	Comparison purpose
Source input	RAW tables loaded from Matillion.[cite:46]	Same RAW tables declared as dbt sources.[cite:52]	Prove same starting point.
Transformation design	Tool-driven workflow with embedded logic.[cite:15] [cite:56]	SQL models split by staging, intermediate, marts.[cite:7] [cite:54]	Show modularity improvement.
Validation	Manual/job-level checks. [cite:26]	Built-in tests and freshness controls.[cite:48][cite:3]	Show governance improvement.
Explainability	Harder to map business rules step by step.	Easier to trace through model lineage and docs.[cite:45] [cite:50]	Show maintainability improvement.
Reusability	Lower reuse across projects.	Higher reuse with macros and intermediate models.[cite:1] [cite:2]	Show scalability improvement.
Delivery output	Legacy transformed tables.	DW/mart business-facing tables.	Show business equivalence.

What to compare later

For each pipeline pair, preserve the following artifacts so you can compare later without rebuilding the story from scratch:

- Source inventory.
- RAW tables used.
- Legacy job screenshots and logic notes.
- dbt model list and lineage screenshots.
- dbt test results.
- Freshness results where applicable.[cite:48]
- Reconciliation SQL and outputs.
- Final sign-off note explaining whether outputs match.

Suggested repository / documentation structure

Use a simple documentation structure so every pipeline is described the same way.

```
/workflow-framework
  /pipeline-simple
    source-definition
    legacy-flow
    future-flow
    validation-checks
    comparison-results
  /pipeline-moderate
    source-definition
    legacy-flow
    future-flow
    validation-checks
    comparison-results
  /shared
    naming-standards
    layer-definitions
    run-sequence
    control-framework
```

How to explain this to others

When presenting, always explain in this order so the audience stays oriented:

1. What is the source?
2. Where does raw data land?
3. How does the old pipeline transform it?
4. How does the new dbt pipeline transform it?
5. Where do tests and freshness checks happen?[cite:48][cite:3]

6. What final table should the business use?

7. How do we know both outputs match?

This order keeps the story grounded in dataflow first, then workflow, then governance, then migration proof.[cite:54] [cite:26]

Minimum deliverables for each pipeline workflow pack

Each demo pipeline should have the following workflow pack prepared:

- One-page source-to-output flow.
- Legacy step map.
- dbt layer map.
- Run sequence checklist.
- Validation checklist.
- Comparison summary.
- Final business output table list.

If both the simple and moderate pipelines use this same pack structure, the team will be able to compare them later in a consistent way and reuse the format for additional pipelines.

[cite:42] [cite:54]